

DSA STUDY BLUEPRINT SERIES

15. Painters Partition Problem

Minimizing the Maximum Time to Paint Boards

Section 02 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Identical to the Book Allocation Problem.
- Divide contiguous boards among K painters to minimize maximum painting time.
- Binary search on time bounds.

Memory & Execution Blueprint

Board Partitions (K = 2 Painters):

[10 | 20 | 30] | [40] $\Rightarrow$ Max Time = **60** vs **40**

C++ Implementation Reference

Standard code layout and syntax

```
int paintBoards(vector& boards, int k) {  
    int s = *max_element(boards.begin(), boards.end());  
    int e = accumulate(boards.begin(), boards.end(), 0);  
    int ans = e;  
    while (s <= e) {  
        int mid = s + (e - s) / 2;  
        if (isValid(boards, boards.size(), k, mid)) {  
            ans = mid; e = mid - 1;  
        } else s = mid + 1;  
    }  
    return ans;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N \log(\text{Sum} - \text{Max}))$
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** Make sure to set the search lower bound to the maximum board length to prevent partition errors.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Binary Search on Answers range checks (e.g. allocating book pages):

Iteration	Check Bounds [Low, High]	Mid Candidate Value	Feasibility Check: <code>isValid(mid)</code>	Search Window Adjustment
1	[90, 203]	mid = 146	True (Students needed $\leq K$)	Shrink search: high = mid - 1 = 145
2	[90, 145]	mid = 117	False (Students needed $> K$)	Expand search: low = mid + 1 = 118

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Minimizing the Maximum:** Used when the answer search space is monotonic (if X is feasible, all values $> X$ are also feasible). Search binary-wise for bounds limits.
- **Feasibility Helpers:** Write custom $O(N)$ linear scan checks to evaluate candidate value viability.
- **Related LeetCode Problems:** #410 Split Array Largest Sum, #1011 Capacity To Ship Packages Within D Days, #875 Koko Eating Bananas.